

Detección de Rostros Sintéticos Generados por StyleGAN3 mediante Vision Transformers

Entrega 2 — Implementación Inicial y Validación Parcial

Jason Barrantes Sánchez · Melany Ramírez Anchía · Walter Bowyer Carpenter · Mauro Espinoza Hernández

Bachillerato en Ingeniería en Ciencia de Datos — Universidad LEAD, San José, Costa Rica

1. Avance Funcional

Se completó la implementación funcional de las Etapas 1 a 5 del pipeline propuesto en la Entrega 1: adquisición de datos con Polars, preprocesamiento paralelo con **ProcessPoolExecutor**, análisis exploratorio avanzado, entrenamiento de un modelo **Vision Transformer (ViT-B/16)** completo sobre GPU real, y evaluación con métricas estándar de clasificación e interpretabilidad. Todo el pipeline se ejecutó de extremo a extremo sobre el clúster Kabré, incluyendo la migración exitosa a la partición GPU **nukwa-l40s** (NVIDIA L40S), no contemplada originalmente en la Entrega 1 por desconocimiento de su disponibilidad.

1.1 Dataset

10,000 Real vs Fake Faces (StyleGAN3), 20,000 imágenes balanceadas (10,000 reales, 10,000 generadas), resolución nativa 1024×1024, 2.11 GB en disco. Split estratificado 80/10/10 (train=16,000, val=2,000, test=2,000).

1.2 Preprocesamiento Paralelo

Split	Imágenes	Tiempo	Throughput
Train	16,000	6.8 s	2,358 img/s
Val	2,000	1.3 s	1,550 img/s
Test	2,000	1.4 s	1,435 img/s

Ejecutado con ProcessPoolExecutor (16 workers) sobre un nodo de la partición nukwa-l40s (Intel Xeon Silver 4416+, 20 núcleos físicos). Las imágenes se almacenan como tensores **uint8** sin normalizar; la normalización ImageNet se aplica en tiempo de carga (ver Sección 4.2), reduciendo el uso de disco en un factor de 4× respecto a almacenar float32 normalizado.

1.3 Entrenamiento del Vision Transformer

Parámetro	Valor
Arquitectura	ViT-B/16 (torchvision), ImageNet-1K preentrenado
Parámetros entrenables	85,800,194 (backbone completo, sin congelar)
Hardware	NVIDIA L40S, 46 GB VRAM, CUDA 12.1, cuDNN 9.1
Precisión	Mixta (AMP, float16)
Optimizador	AdamW, lr=1e-4, weight_decay=0.01
Batch size	32

Épocas	15
Tiempo total	10.75 minutos
Throughput promedio (train)	565.1 img/s

2. Organización del Código

El código se organiza en módulos independientes con responsabilidad única, siguiendo el principio de separar cómputo pesado (scripts standalone ejecutables vía SLURM) de exploración/visualización (notebooks Jupyter):

Archivo	Responsabilidad
src/preprocessing_worker.py	Función worker de preprocesamiento + clases Dataset de PyTorch (picklables para multiprocessing)
run_preprocessing.py	Preprocesamiento paralelo standalone (split + ProcessPoolExecutor)
train_vit.py	Entrenamiento del ViT con checkpointing, AMP, y monitor de GPU
submit_train_vit.sh	Job SLURM para entrenamiento desatendido (partición nukwa-l40s)
notebooks/01_*.ipynb	Adquisición de datos, inventario Polars, EDA inicial
notebooks/02_*.ipynb	EDA avanzado: FFT, features visuales, detección de duplicados
notebooks/03_*.ipynb	Evaluación del modelo, interpretabilidad, clasificador de fotos

Esta separación responde directamente al límite de 4 horas por trabajo en las particiones GPU de Kabré: el entrenamiento (proceso largo, no interactivo) corre como trabajo *sbatch* desatendido, mientras que la evaluación (rápida, exploratoria) corre en sesiones interactivas de Jupyter.

3. Validación Inicial

Se evaluó el modelo entrenado sobre el conjunto de test (2,000 imágenes, disjunto de entrenamiento y validación):

Métrica		Valor	
Accuracy		0.8700	
Precision		0.8376	
Recall		0.9180	
F1-Score		0.8760	
AUC-ROC		0.9438	

Clase	Precision	Recall	F1-Score
Fake	0.91	0.82	0.86
Real	0.84	0.92	0.88

El modelo detecta rostros reales con mayor recall (92%) que rostros sintéticos (82%), resultado consistente con el objetivo de diseño de StyleGAN3 — arquitectura específicamente formulada para eliminar artefactos de aliasing y maximizar la dificultad de detección automática. Como validación cualitativa adicional, se implementó una función de clasificación de fotografías arbitrarias fuera del dataset; una foto personal de uno de los integrantes del equipo fue clasificada correctamente como **REAL con 99.99% de confianza**, confirmando generalización más allá de las condiciones controladas del dataset de origen.

3.1 Interpretabilidad

Se extrajeron mapas de atención del token [CLS] mediante un context manager que intercepta temporalmente el `forward()` de `nn.MultiheadAttention` (torchvision no expone estos pesos por defecto, al invocar la atención con `need_weights=False` por eficiencia). Esto permite visualizar en qué regiones de la imagen se enfoca el modelo al tomar

su decisión de clasificación, sin modificar la arquitectura del modelo de forma permanente.

4. Gestión de Problemas

Durante la implementación se identificaron y resolvieron ocho obstáculos técnicos significativos, documentados aquí con su solución aplicada:

1. Acceso SSH bloqueado. El puerto 22 hacia kabre.cenat.ac.cr resultó bloqueado por firewall institucional desde redes externas a CeNAT. Se resolvió utilizando el Shell Access de Open OnDemand, funcionalmente equivalente a una sesión SSH nativa (mismo SLURM, mismo filesystem, misma cuenta).

2. Cuota de home insuficiente. El directorio home de Kabré tiene un límite fijo de 10 GB, insuficiente para el entorno virtual (con PyTorch+CUDA) más el dataset. Se migró el proyecto completo al filesystem Lustre /data, con cuota personal de 20 GB.

3. Ausencia de GPU en particiones estándar. Las particiones CPU por defecto (kura, kura-wide, kura-long, kura-debug) no exponen GPU alguna (440 CPUs, 0 GPUs). Mediante inspección del selector de particiones de Open OnDemand se identificaron las particiones nukwa-v100 y nukwa-l40s, confirmadas con nvidia-smi.

4. Sintaxis GRES no soportada. La sintaxis estándar `--gres=gpu:1` de SLURM produce el error 'Invalid generic resource specification' en Kabré, ya que los nodos reportan `Gres=(null)`. La asignación de GPU se logra simplemente seleccionando la partición correspondiente, sin bandera `--gres` adicional.

5. Escritura de disco interrumpida por cuota. Un primer intento de preprocesamiento almacenando tensores float32 normalizados (≈ 12 GB para el dataset completo) excedió la cuota de 20 GB a mitad de la escritura del último chunk, con `OSError` confirmando el corte exacto en bytes. Se rediseñó el formato de almacenamiento a uint8 crudo (2.9 GB total), aplicando la normalización ImageNet en tiempo de carga dentro de la clase Dataset.

6. Límite de 4 horas por trabajo GPU. Las particiones nukwa-v100/nukwa-l40s imponen `MaxTime=04:00:00` por trabajo. `train_vit.py` implementa checkpointing por época (`vit_last.pt` para continuar, `vit_best.pt` según mejor AUC) y una bandera `--resume`, permitiendo reanudar el entrenamiento sin pérdida de progreso si un trabajo es interrumpido.

7. Sobreajuste del modelo. A partir de la época $\sim 10-12$ se observó divergencia entre las métricas de entrenamiento (`accuracy > 98%`) y validación (AUC estable $\approx 0.94-0.95$, `loss` creciente). La estrategia de guardar el checkpoint según el mejor AUC-ROC de validación (no la última época) selecciona automáticamente el punto de mejor generalización (época 11), mitigando el problema sin intervención manual adicional.

8. Intento previo inviable en CPU. Un primer intento del equipo, ejecutado en una máquina sin GPU, fue interrumpido manualmente durante la primera época de entrenamiento debido al costo computacional de ajustar 85.8M de parámetros sin aceleración por hardware. La migración a la GPU L40S de Kabré resolvió este cuello de botella, permitiendo completar 15 épocas en menos de 11 minutos.

5. Próximos Pasos (Entrega 3)

- Generación de rostros sintéticos nuevos con el generador preentrenado oficial de StyleGAN3 (NVIDIA), para evaluar la generalización del clasificador más allá del dataset fijo de Kaggle.
- Cuantificación formal de speedup y eficiencia paralela (comparación serial vs. paralelo en preprocesamiento, siguiendo la metodología planteada en la Entrega 1).
- Desarrollo del dashboard interactivo (Plotly/Dash) para explorar resultados y métricas de rendimiento computacional.
- Análisis de escalabilidad fuerte variando el tamaño de muestra en el preprocesamiento paralelo.
- Redacción del informe técnico final en formato IEEE completo, integrando resultados de todas las etapas.

Repositorio del proyecto: código fuente, notebooks, scripts de ejecución y resultados experimentales disponibles en GitHub (ver README.md para instrucciones completas de reproducción).